

Nexus ERP Fase 2

Para que o seu sistema saia do “papel” (código de simulação) e funcione em um cenário real, precisamos conectar o **Endpoint** (a porta de entrada) ao **Banco de Dados Real** (o cofre de informações).

Aqui está a explicação de como essa engrenagem funciona e o código para realizar essa conexão.

1. O que é o quê nessa engrenagem?

- * **Banco de Dados Real (PostgreSQL):** É um software rodando em um servidor que guarda as tabelas permanentemente. Mesmo que você desligue o computador, os dados estão lá.
 - * **Endpoint (API):** É um endereço (ex: `meusistema.com/vendas`) que recebe ordens. Ele é o intermediário.
 - * **O “Driver” (A Ponte):** Para o Python falar com o PostgreSQL, usamos uma biblioteca chamada `SQLAlchemy` ou `psycopg2`. Ela traduz os comandos do código para a linguagem do banco.
-

2. O Fluxo de Trabalho (O caminho do dado)

1. **O Usuário** clica em “Salvar Venda” na tela.
 2. **O Front-end** envia um pacote de dados (JSON) para o seu **Endpoint**.
 3. **O Endpoint** recebe, valida (vê se tem estoque e crédito) e abre uma **Transação** com o Banco de Dados.
 4. **O Banco de Dados** grava a informação e responde “Ok”.
 5. **O Endpoint** responde para o usuário: “Venda realizada com sucesso!”.
-

3. Código Prático: Conectando o Endpoint ao Banco Real

Para este exemplo, vamos usar o **SQLAlchemy**, que é o padrão de mercado para garantir a **Qualidade (SAP)**, pois ele evita erros de segurança (como SQL Injection).

Passo A: Instalar as ferramentas

No terminal: `pip install fastapi uvicorn sqlalchemy psycopg2-binary`

Passo B: O Código de Conexão e Gravação

```
` `` python
```

```
From fastapi import FastAPI, Depends, HTTPException
```

```
From sqlalchemy import create_engine, Column, String, Float, DateTime
```

```
From sqlalchemy.ext.declarative import declarative_base
```

```
From sqlalchemy.orm import sessionmaker, Session
```

```
Import datetime
```

```
Import uuid
```

1. CONFIGURAÇÃO DO BANCO DE DADOS (O 'Cofre')

Substitua pelos seus dados reais: usuário, senha, endereço do servidor, nome do banco

```
DATABASE_URL = "postgresql://usuario:senha@localhost:5432/nexus_erp"
```

```
Engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

2. DEFINIÇÃO DA TABELA (O 'Modelo' no Python)

Class SalesDB(Base):

__tablename__ = "sales"

Id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))

Customer_name = Column(String)

Total_value = Column(Float)

Created_at = Column(DateTime, default=datetime.datetime.utcnow)

Criar a tabela no banco real se não existir

Base.metadata.create_all(bind=engine)

3. ENDPOINT (A 'Porta de Entrada')

App = FastAPI()

Função para abrir e fechar a conexão com o banco automaticamente

Def get_db():

Db = SessionLocal()

Try:

Yield db

Finally:

Db.close()

@app.post("/sales/save")

Def save_sale(customer: str, value: float, db: Session = Depends(get_db)):

LÓGICA DE NEGÓCIO (Qualidade)

If value <= 0:

Raise HTTPException(status_code=400, detail="Valor da venda inválido")

GRAVAÇÃO NO BANCO REAL

```
New_sale = SalesDB(customer_name=customer, total_value=value)
```

```
Db.add(new_sale) # Adiciona
```

```
Db.commit()    # Salva permanentemente (O 'Commit' do SAP)
```

```
Db.refresh(new_sale) # Atualiza os dados
```

```
Return {"status": "Venda gravada no Banco Real", "id": new_sale.id}
```

```
...
```

4. Por que isso é importante para o seu sistema?

A. Atomicidade (Tudo ou Nada)

No Nexus, quando você faz uma compra, o sistema deve:

1. Aumentar o estoque.
2. Criar a conta a pagar.
3. Gerar o lançamento contábil.

****Com o Banco de Dados Real****, se o passo 3 falhar, o sistema cancela o 1 e o 2 automaticamente (Rollback). Isso é a ****Robustez SAP****.

B. Performance

Um banco real como o PostgreSQL aguenta milhões de vendas por dia sem ficar lento, garantindo que a ****Rentabilidade**** da empresa não seja prejudicada por um sistema travado.

C. Segurança

O Endpoint garante que ninguém mexe no banco de dados sem permissão. Você coloca uma camada de login (OAuth2) no Endpoint e o banco de dados fica “escondido” atrás dele.

5. O que você precisa para testar isso agora?

1. **Instalar o PostgreSQL** no seu computador (ou usar um serviço na nuvem como a AWS ou Supabase).
2. **Configurar a URL de conexão** (`DATABASE_URL`) no código acima.
3. **Executar o código** e usar uma ferramenta como o **Postman** ou a própria tela de documentação do FastAPI (`http://localhost:8000/docs`) para enviar os dados.

Você quer que eu ajude a criar o **script de automação** que já instala essas dependências e sobe o banco de dados para você começar a programar de imediato?